

CDWIA v2 — Production Architecture, Flows & Tooling Deep Dive

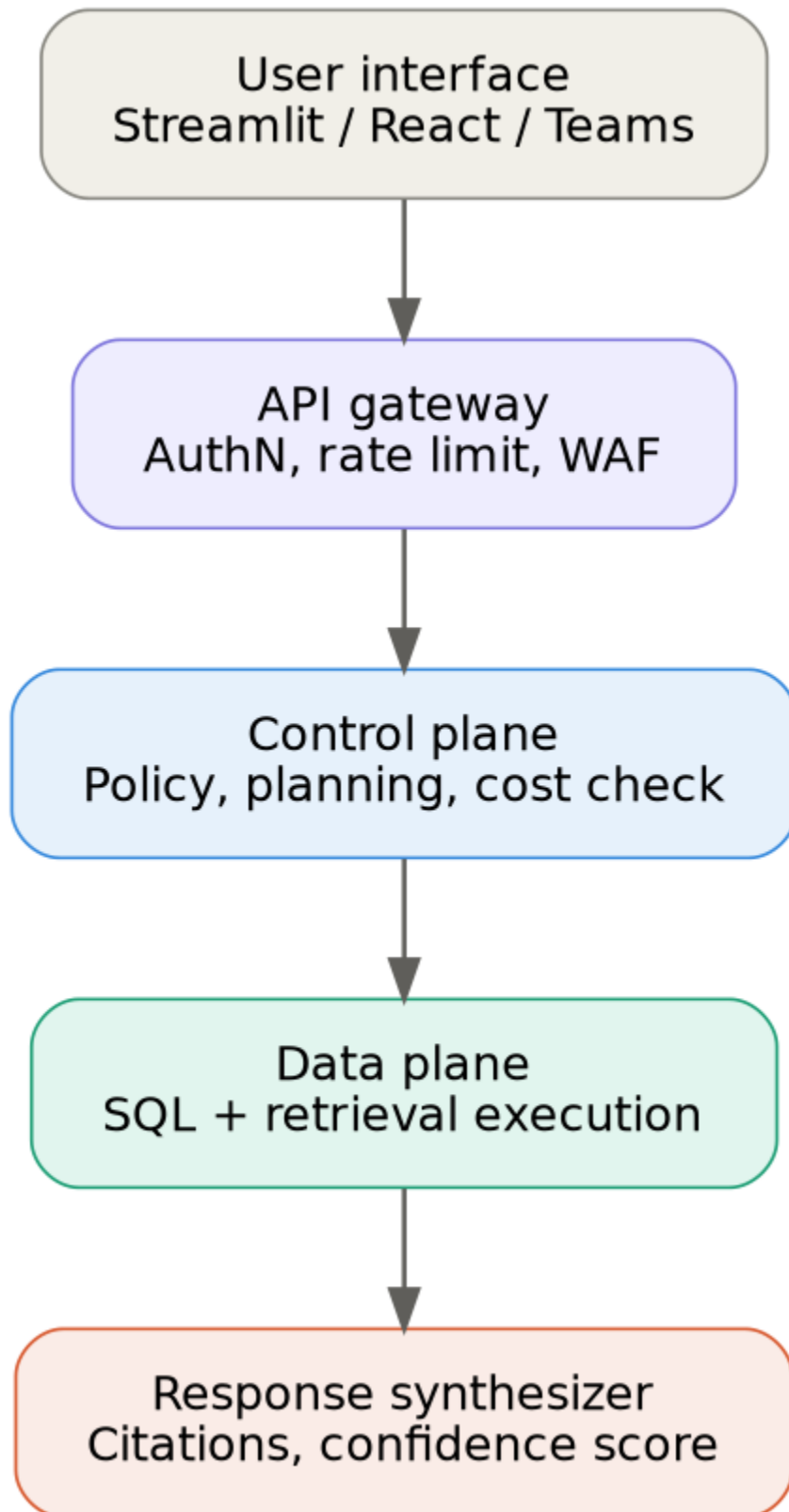
Cloud Data Warehouse Intelligence Assistant

1. Executive summary

This report extends the original CDWIA proposal into a production-grade design. It restructures the system around a **control plane / data plane split**, closes every gap flagged in the original critical evaluation (missing gateway, model router, semantic layer, cost estimator, policy engine, retries, evaluation pipeline, ingestion pipeline, and more), and documents *why* each tool was chosen over its alternatives. This version goes one level deeper than the initial architecture review: it walks the hardest case — a **hybrid query** — step by step, and shows the actual **decision logic** behind SQL safety validation.

2. Architecture, level 1 — system context

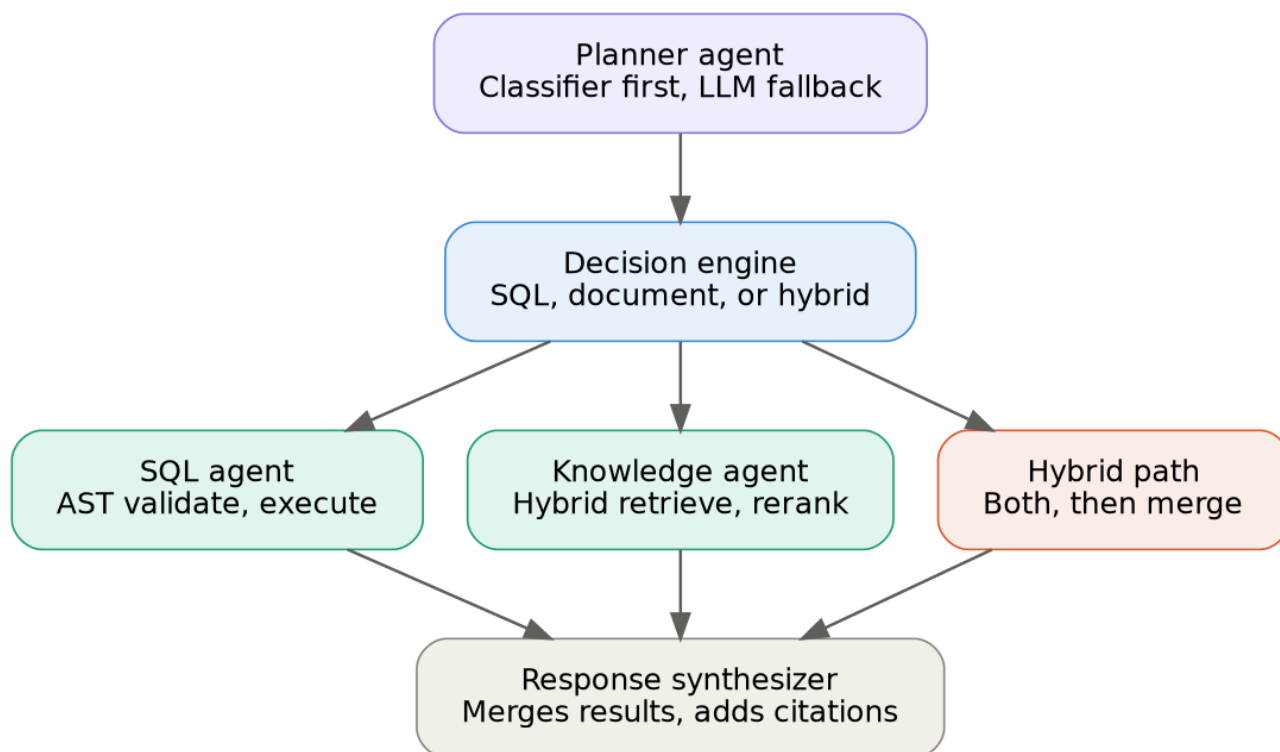
The outside-in view: a request moves from the user through the gateway into the control plane, executes in the data plane, and returns as an explained, cited answer.



The key design choice here is that **the control plane sits before the data plane in the request path**, not alongside it. Authorization and cost checks happen *before* any SQL is generated or any vector search runs — this avoids paying for LLM/compute work on requests that would be denied or rejected anyway.

3. Architecture, level 2 — decision engine flow

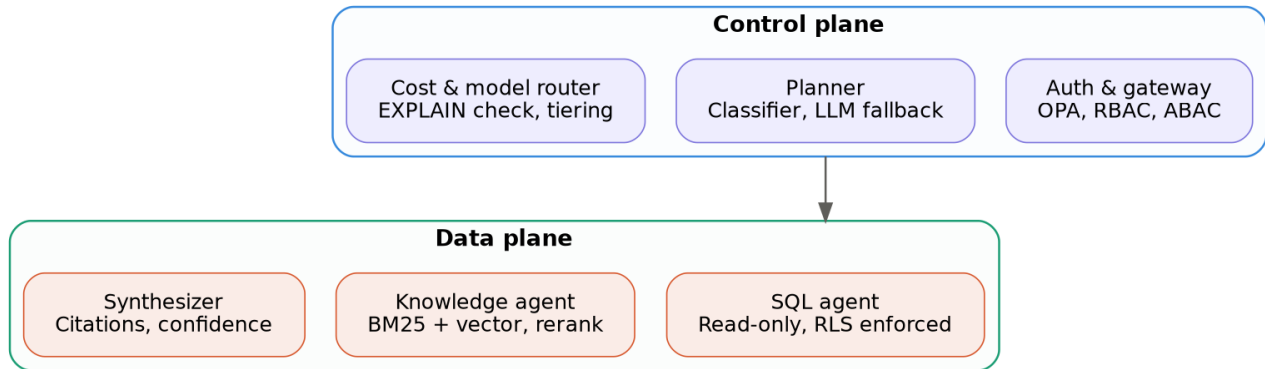
Zooming into the control plane's core responsibility: classify the question, route it, execute one of three paths, and converge on a single synthesizer.



A **deterministic classifier runs before the LLM**, not after. Most questions ("which team spent the most on EC2 last month") are structurally obvious and can be classified in milliseconds by a lightweight model; the LLM planner is only invoked as a fallback when classifier confidence is low. This is the single highest-leverage cost and latency optimization in the whole system, since it removes an LLM call from the majority of traffic.

4. Architecture, level 3 — control plane vs data plane

The containment view: why the split exists, and what lives on each side.

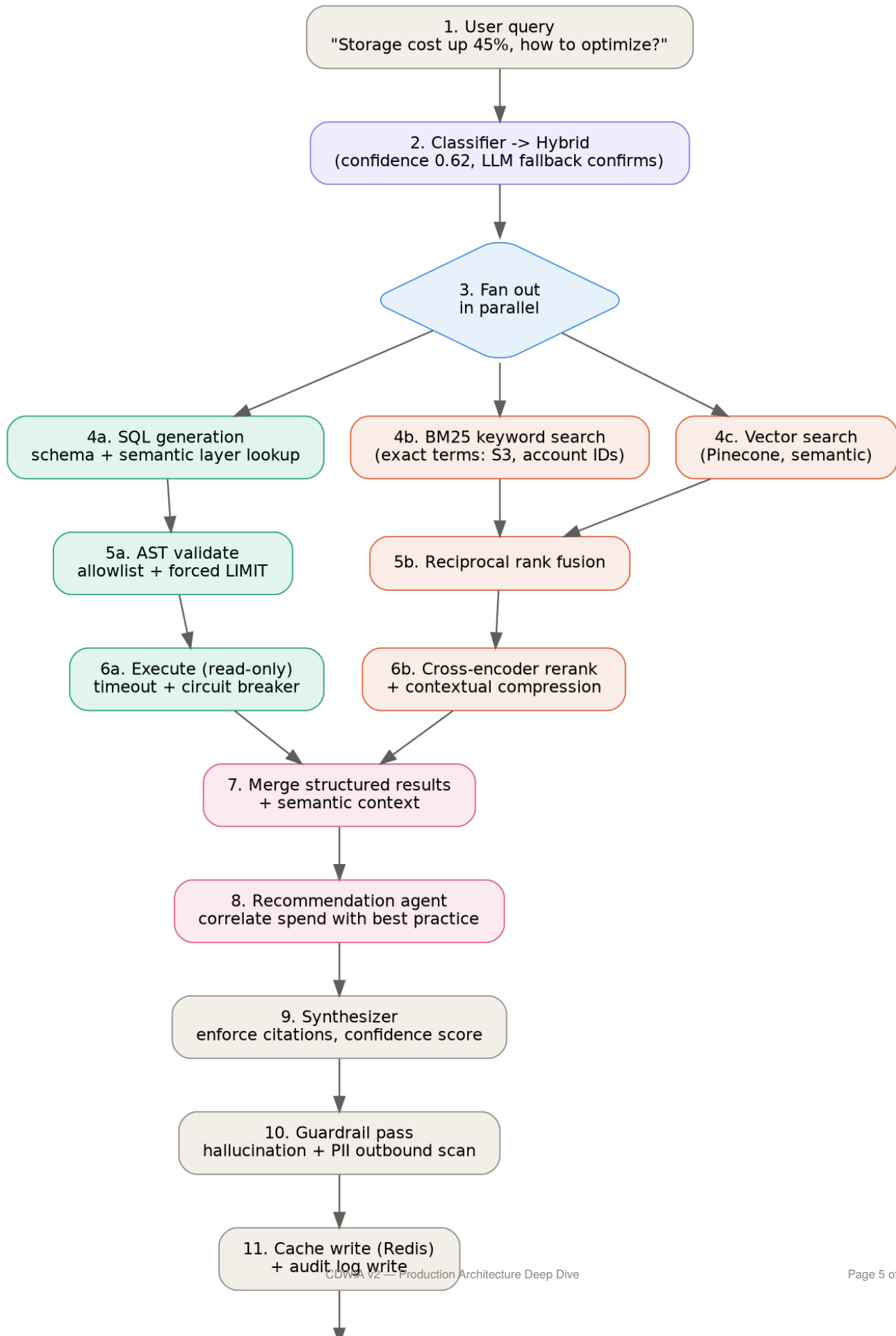


- **Control plane** = decisions. Low throughput, must be correct, changes slowly (policy, routing logic).
- **Data plane** = execution. High throughput, must be fast and cheap, scales horizontally and independently of the control plane.

Separating them means a policy change (e.g. a new OPA rule) can ship without touching the SQL execution path, and the data plane can autoscale under load without the control plane needing to scale in lockstep.

5. Deep dive — the hybrid query flow, step by step

Hybrid queries are the hardest case: the system must generate SQL *and* retrieve documents *and* correlate them into one coherent recommendation. This is the flow the original design described only at a high level ("Generate SQL + Retrieve Documents → Merge → LLM Synthesizes"). Here is the full path, including validation and guardrail steps that were missing from the original design:



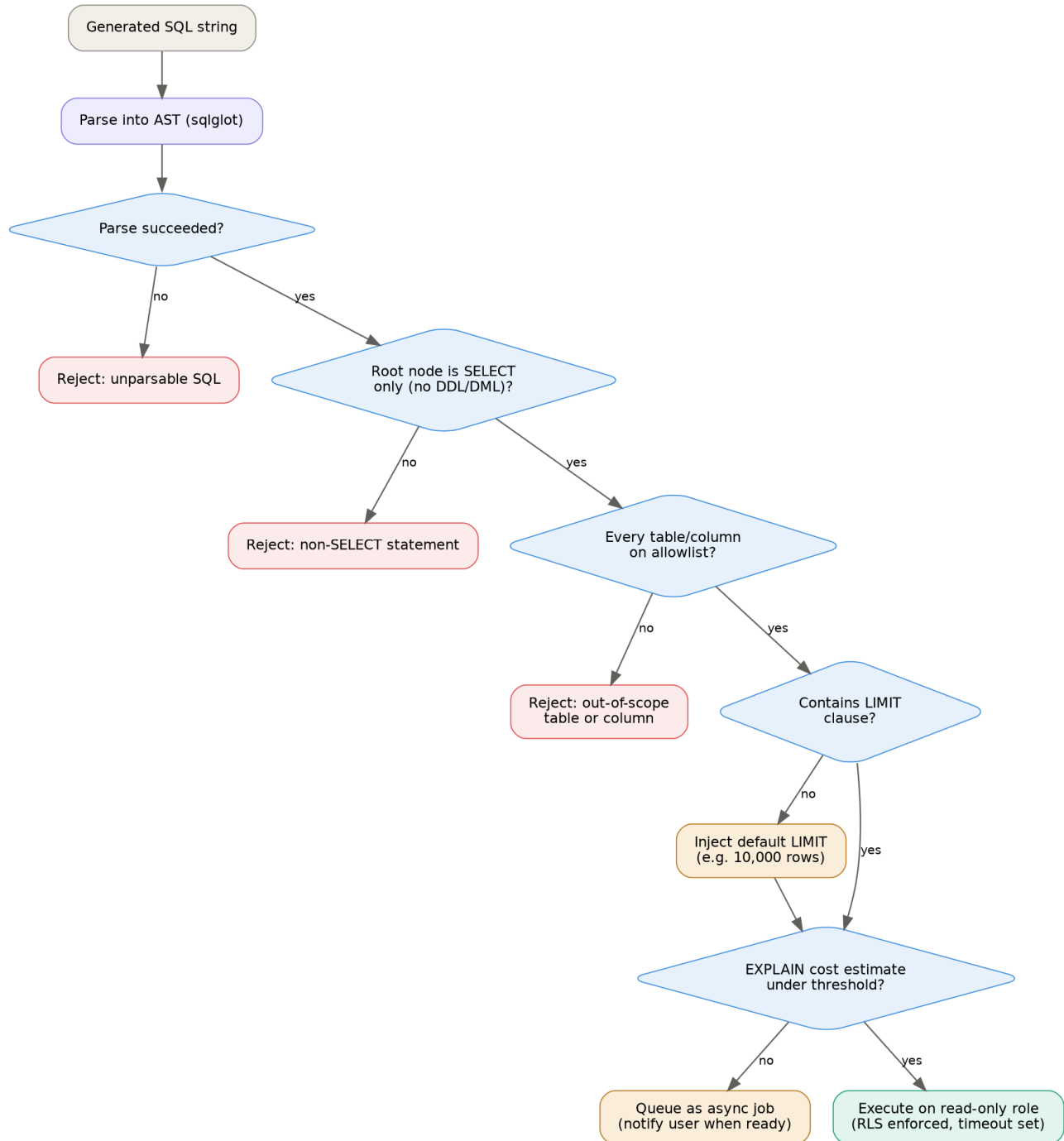
Walkthrough, using the example *"Our storage cost increased by 45%. How can we optimize it?"*:

1. **Classification.** The deterministic classifier scores this around 0.62 confidence for "hybrid" — ambiguous enough to trigger the LLM planner fallback, which confirms hybrid (the question needs both a "what happened" answer from data and a "what to do" answer from documentation).
2. **Fan-out.** SQL generation and document retrieval run **in parallel**, not sequentially — this is a latency optimization; there's no dependency between the two paths at this stage.
3. **SQL path.** Schema/semantic-layer lookup resolves "storage cost" to the correct physical tables, SQL is generated, then passed through the **AST validator** (Section 6) before executing on a read-only role.
4. **Retrieval path.** BM25 catches exact terms like "S3" or specific bucket/account identifiers; vector search catches the semantic intent ("optimize storage cost"); results are fused via reciprocal rank fusion, then reranked and compressed to the most relevant sentences.
5. **Merge.** Structured results (which accounts/services drove the 45% increase) and semantic context (relevant FinOps guidance) are combined by the recommendation agent, which correlates the two rather than just concatenating them.
6. **Synthesis and guardrails.** The response synthesizer enforces that every claim in the final answer maps to either a SQL result or a retrieved citation — an unsupported claim is not allowed through. A guardrail pass then re-checks for hallucination and PII leakage in the *outbound* text, not just the inbound query.
7. **Cache and audit.** The result is cached (keyed on normalized query + the requester's data scope, so it's safe to reuse across users in the same business unit) and an immutable audit log entry is written before the response reaches the user.

This is also where most of the original design's implicit assumptions become explicit: "LLM synthesizes" (from the original diagram) is really three separate, auditable steps — merge, synthesize, guardrail-check — each of which can fail independently and needs its own error handling.

6. Deep dive — the AST SQL validator

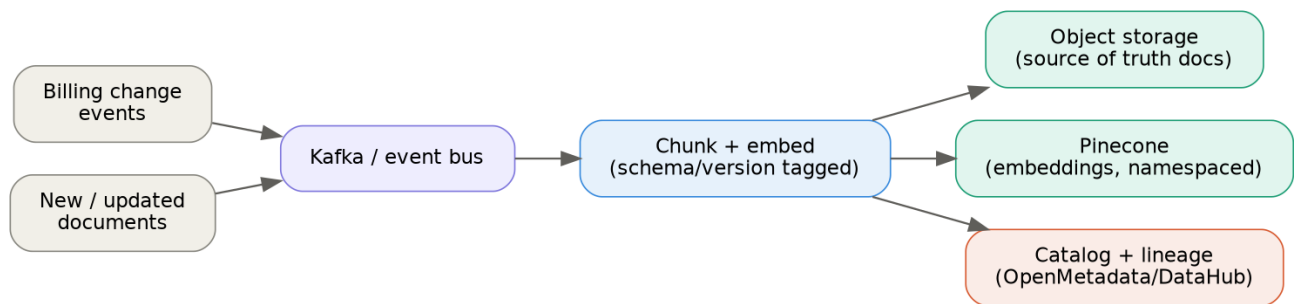
The original design listed "SQL Validator" as a single box with no defined behavior. This is the actual decision logic that closes that gap — the point of using an **AST parser (e.g. `sqlglot`)** rather than regex/string matching is that every check below operates on the *parsed structure* of the query, which cannot be bypassed by comments, string-escaping tricks, or nested subqueries the way a string-match filter can.



Each rejection path returns a specific, loggable reason (useful both for debugging and for detecting repeated attempts to probe the boundary, which is itself a signal worth alerting on). Two of the five checks don't reject outright — a missing `LIMIT` gets one silently injected, and a query that's too expensive gets queued asynchronously rather than executed inline, so the user still gets an answer, just not instantly.

7. Async path — ingestion pipeline

Everything above happens synchronously, in the request path. New billing data and new documents need a separate, asynchronous pipeline so ingestion never blocks or slows down live queries — this was entirely absent from the original design.



Kafka (or a managed equivalent) decouples the arrival of new data from its processing. Every chunk is tagged with a schema/document version at ingestion time, which is what makes it possible to later answer an audit question like *"what did the assistant know when it gave this recommendation on this date"* — a requirement the original design had no mechanism for at all.

8. Tool decisions — chosen vs rejected, with the deciding advantage

8.1 Orchestration & routing

Component	Chosen	Advantage over alternatives	Rejected
Agent orchestration	LangGraph	Explicit state machine supports cycles (retries, re-planning); plain chains can't branch cleanly	LangChain sequential chains, CrewAI, AutoGen
Query routing	Deterministic classifier + LLM fallback	Sub-50ms for common cases, no LLM cost on obvious questions, deterministic and testable	LLM-only planning
Tool interface	MCP tool abstraction	Standardizes tools so non-LangGraph clients can reuse them; future-proofs new integrations	Bespoke LangChain tool wrappers per integration

8.2 SQL layer

Component	Chosen	Advantage over alternatives	Rejected
SQL validation	AST parser (sqlglot) + allowlist	Parses real syntax tree; can't be bypassed by comments/encoding tricks the way regex can	Regex/string-match filtering
DB access	Read-only role + row-level security	Database itself blocks writes/cross-tenant reads even if app logic has a bug	Full-access connection with app-only checks
Cost control	EXPLAIN-based estimator + forced LIMIT + timeout	Stops a legitimate-looking but catastrophic query before it locks the warehouse	No pre-execution cost check

8.3 Retrieval (RAG)

Component	Chosen	Advantage over alternatives	Rejected
Retrieval strategy	Hybrid BM25 + vector	Catches exact-match terms (SKUs, account IDs, service codes) vector search misses	Vector-only search
Precision boost	Cross-encoder / Cohere rerank	Meaningfully improves precision@k over raw bi-encoder ranking	No reranking pass
Context handling	Contextual compression	Lower token cost, less irrelevant text for the model to hallucinate from	Full chunk/document dumps into context

8.4 Governance & policy

Component	Chosen	Advantage over alternatives	Rejected
Access control	OPA (ABAC + RBAC)	Policy is declarative, versioned, testable independent of app code	Hardcoded role checks in application logic
Schema/metric definitions	Semantic layer + data catalog	Metric meaning is a governed artifact, not re-derived by LLM guesswork each call	Pure information_schema introspection per query
Multi-tenancy	RLS + vector namespace + policy, all three layers	Defense in depth — one layer's bug doesn't leak cross-tenant data	Single-layer isolation (DB only)

8.5 Reliability & observability

Component	Chosen	Advantage over alternatives	Rejected
Model resilience	Model router + fallback chain	Cost-tiered by task, survives a provider outage	Single model for every task
Failure handling	Retries + circuit breakers + async queue	Degrades gracefully instead of cascading failure on one slow dependency	Synchronous-only, no breaker
Tracing	OpenTelemetry + LangSmith + Prometheus/Grafana	Full-stack trace (infra) plus LLM-specific trace (prompts/tokens)	LangSmith alone (no infra visibility)
Quality gate	Golden dataset + offline eval in CI/CD	Catches prompt/model regressions before they reach users	Manual QA only

9. Security, guardrails & PII — hardened review

Layer	Control	Detail
Transport	TLS everywhere	mTLS between internal services where feasible
Identity	OAuth2/OIDC via Azure AD/Okta	Short-lived tokens, refresh rotation
Authorization	RBAC + ABAC (OPA) + RLS	Enforced before planning, not just at the DB layer
SQL safety	AST validation + allowlist + read-only role + LIMIT + timeout	No string/regex-only filtering
Prompt safety	Injection detector on input, citation enforcement + hallucination detector on output	Both directions checked, not just input
Secrets	AWS Secrets Manager / HashiCorp Vault	No secrets in code/config; rotated
Encryption	AES-256 at rest, TLS 1.2+ in transit	Field-level encryption for highly sensitive columns
Audit	Immutable audit log (query, SQL run, docs cited, model/prompt version)	Required for SOC2/enterprise compliance

PII handling, explicitly:

1. **Inbound scan** — the user's query is scanned/masked for PII before it reaches any third-party LLM API.
 2. **Ingestion scan** — documents are scanned for PII/sensitivity at ingestion and tagged; retrieval respects the requester's clearance.
 3. **Outbound scan** — generated responses are scanned before being returned, in case a SQL join surfaces PII a role shouldn't see.
 4. **Data minimization** — PII-bearing columns are excluded from the semantic layer's exposed metric definitions entirely, not filtered only at query time.
-

10. Reliability, scalability & cost — summary

- **Horizontal scaling**: stateless services autoscale on Kubernetes by queue depth and latency, not just CPU.
 - **Graceful degradation**: circuit breakers mean a Pinecone outage degrades to "SQL-only answers, no citations" rather than a full outage.
 - **DR**: cross-region backups (PITR for Postgres, index snapshots for Pinecone), target RPO ≤ 15 min / RTO ≤ 1 hr for the data plane.
 - **Cost levers**: model routing by task complexity, the deterministic classifier removing LLM calls from most requests, semantic/result caching, the query cost estimator preventing runaway warehouse compute, and context compression reducing token spend per call.
 - **Recommendation**: instrument cost-per-query as a first-class metric, broken down by SQL vs RAG vs hybrid path — a FinOps tool should itself be FinOps-managed.
-

11. Phased delivery roadmap

Phase 1 — Foundations: API gateway, AuthN/AuthZ + OPA, read-only SQL role + RLS, AST SQL validator, basic observability, audit logging.

Phase 2 — Core intelligence: deterministic classifier, hybrid retrieval + reranker, semantic layer/catalog, model router + fallback, Redis caching.

Phase 3 — Reliability & scale: circuit breakers, async queue, DR strategy, multi-tenant isolation hardening, load-testing gates in CI/CD.

Phase 4 — Governance & continuous improvement: golden dataset + offline eval pipeline, human feedback loop, lineage/catalog maturity, cost dashboards, MCP tool abstraction rollout.

12. Open questions for stakeholders

1. Expected query volume and peak concurrency — determines Pinecone/OpenSearch clustering and Postgres read-replica count.
 2. Data residency/compliance regime (SOC2, GDPR, FedRAMP) — determines DR targets and whether OPA policies need region-specific variants.
 3. Tenant model — single enterprise with internal business units, or true multi-customer SaaS? Changes how aggressively isolation needs to be tested and certified.
 4. Budget ceiling per query/month — needed to tune the model router's cost/quality tradeoff concretely.
-

This report supersedes the architecture and tooling sections of the original CDWIA proposal. The original's "Key Features" and "Expected Outcomes" remain valid as business goals; this document is about how to deliver them at production grade.